Tejas Upmanyu

Follow 40 Followers About

Setting Up Unit Tests In React-Typescipt With Jest & Enzyme

A quick guide to getting your test cases running with Jest & Enzyme in React-Typescript. 😎

Tejas Upmanyu Dec 22, 2018 · 5 min read

your code. Why should you write test cases is an age-old but gold question, which I have tried to answer in this post, do check it out.

Typescript has more presence compared to flow when it comes to maintaining type-safety in React codebase. Type-safety might seem like an obtrusion at first but is really important when you have mammoth project with a lot of developers working. Type-safe implementations lead to better readability and lesser bugs in general.

Since you are looking for setting up test environment in React-Typescript using **Jest & Enzyme**, let's get cracking.

Step 0. Pre-requisites

You probably already have a react-typescript in shape, if so please feel free to go to next steps. In case you are just starting fresh and want to take test setup proactively, go ahead and install React and Typescript with your favourite package manager.

```
npm i react react-dom typescript

# If you use yarn, then -

yarn add react react-dom typescript
```

Step 1. Install Jest ⚡

Jest which is an amazing Javascript testing Utility by Facebook. In this tutorial, we'll be using Jest as our test runner. If you are building your React app with `Create-React-App` (referred as CRA), then rest assured as Jest gets pre-installed, otherwise, go ahead and install Jest.

```
npm i jest @types/jest ts-jest --dev

# if you use yarn, then -

yarn add jest @types/jest ts-jest -D
```

with our code.

- We Install `Jest`. Our test runner.
- Install types for `Jest` by installing `@types/jest`
- `ts-jest` is a typescript pre-processor for Jest. `ts-jest` allows for converting (transpiling) our typescript code and also paves way for source maps out of the box.
- We use `--dev` flag or `-D` to ensure that these packages get installed as developer dependencies and not as usual packages. Since we do not require these modules on production environment and saving them as dev dependencies omits them from production build, decreasing production bundle size.

Step 2. Configure Jest Installation 🛠

In previous step, we installed Jest and other supporting packages for typescript environment. If you bootstrapped your React application with CRA then Jest comes pre-configured out of the box but in case you haven't used CRA. It's time to configure our Jest installation indicating It the location of our test files and how to look for them. Create a new file named **jest.config.js** at root location of your project and paste the following code as is, inside it.

```
module.exports = {
roots: ["<rootDir>/src"],
transform: {
"^.+\\.tsx?$": "ts-jest"
},
testRegex: "(/__tests__/.*|(\\.|/)(test|spec))\\.tsx?$",
moduleFileExtensions: ["ts", "tsx", "js", "jsx", "json", "node"]
}
```

assume this is as the case and specify this using the `roots` option.

- The `transform` config just tells Jest to use `ts-jest` for ts / tsx files.
- The `testRegex` tells Jest to look for tests in any `__tests__` folder AND also any files anywhere that use the `(.test|.spec).(ts|tsx)` extension e.g. `message.test.tsx` etc.
- The `moduleFileExtensions` tells Jest to recognize our file extensions. This is needed as we add ts/tsx into the defaults (`js|jsx|json|node`).

Step 3. Setup Enzyme

Since you are here, I can safely assume you know about Enzyme. Enzyme is an amazing React testing utility by the good folks at Airbnb with great React DOM support making it easier to assert, manipulate and traverse React components while testing.

```
npm i enzyme @types/enzyme enzyme-to-json enzyme-adapter-react-16 --- dev

# or if you are using yarn -

yarn add enzyme @types/enzyme enzyme-to-json enzyme-adapter-react-16 -D
```

So, there is a lot more here besides installing Enzyme. Let's go through all the packages we've installed and what are they gonna help us with.

- `@types/enzyme` gets us type definitions for Enzyme.
- `enzyme-to-json` is a very popular snapshot serializer, particularly employed for snapshot testing for which Jest is quite famous. It serializes the snapshot to JSON for easier comparison.
- `enzyme-adapter-react-16` is an adapter for your React version. Since I am using React 16.x, we install `enzyme-adapter-react-16`

snapshot testing in your application.

```
module.exports = {

// ...at the end, other parts as before.

snapshotSerializers: ["enzyme-to-json/serializer"],
setupTestFrameworkScriptFile: "<rootDir>/src/setupEnzyme.ts"
}
```

So, let's get behind what these new couple of lines mean —

- With `snapshotSerializers: ["enzyme-to-json/serializer"]` we set snapshot serializer for Jest to be the `enzyme-to-json` serializer we installed in the previous step.
- With `setupTestFrameworkScriptFile: "<rootDir>/src/setupEnzyme.ts"` we tell Jest where to look for our setup tests file. A file which sets up our test cases to begin with.

Step 5. Creating Test Setup File. ✔

As you might be wondering — we did tell Jest to look for our setup test file in the previous step but we haven't created one yet. So let's go ahead and make one. Navigate to your `src` folder and create a file named **setupEnzyme.ts** and write the following code inside It —

```
import * as Enzyme from "enzyme";

import * as Adapter from "enzyme-adapter-react-16";

Enzyme.configure({ adapter: new Adapter() });
```

Save and that creates our test setup file for Enzyme.

Step 6. Mention How To Run Tests 🤜

`scripts:` object for a key named `test` if there isn't one, go ahead and create one like —

```
scripts : {
...
"test": "jest",
"test:watch": "jest --watch"
}
```

Now just head over to terminal and go into your project root directory, type in `npm run test` and press enter to see your test cases running!

In Case you are using aliases for directories inside your project...

Often times we have aliases used heavily in a project. By aliases I mean mapping of specific folders in your project by a keyword or path. Here's an example —

Instead of importing a file from `../../../components/MainComponent/` we can just define an alias for `/components` directory in **tsconfig.json** under paths object;

```
"paths": {
"components/*": ["src/components/*"]
}
```

Now you can just type in components and VSCode will provide suggestions for importing from `components/*` . Nice huh! 🤓

Now if you have aliases setup, which is the case with most large scale projects. You just need to add a moduleMapper key and provide paths for your aliases to Jest, inside **jest.config.js** like this —

```
moduleNameMapper: {

"^components(.*)$": "<rootDir>/src/components$1",
```

Head over to Jest documentation for more information. 😍

> *https://jestjs.io/docs/en/getting-started*

Hope this helps you in setting up your test cases in no time. Have a great time writing great test cases and hence writing solid code! 😇

JavaScript React Typescript Jest Enzyme